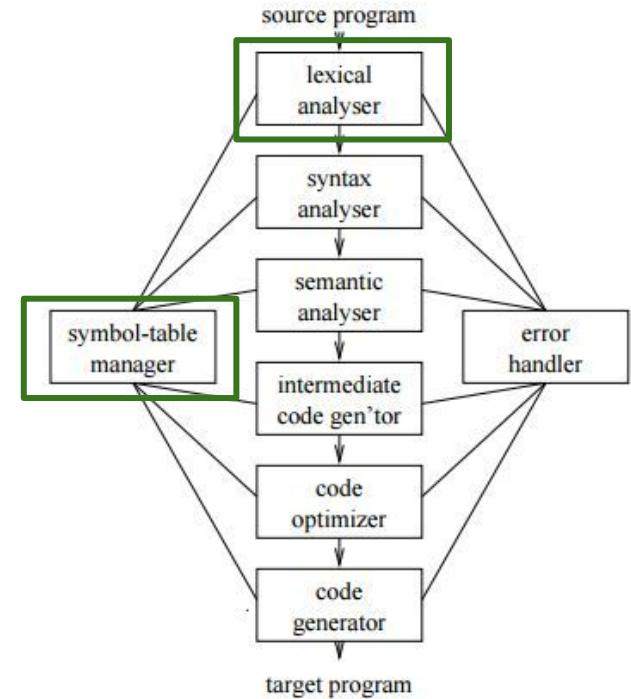


# Syntax and Semantic Analysis with Antlr

Md Irtiaz Kabir  
Lecturer  
CSE, BUET

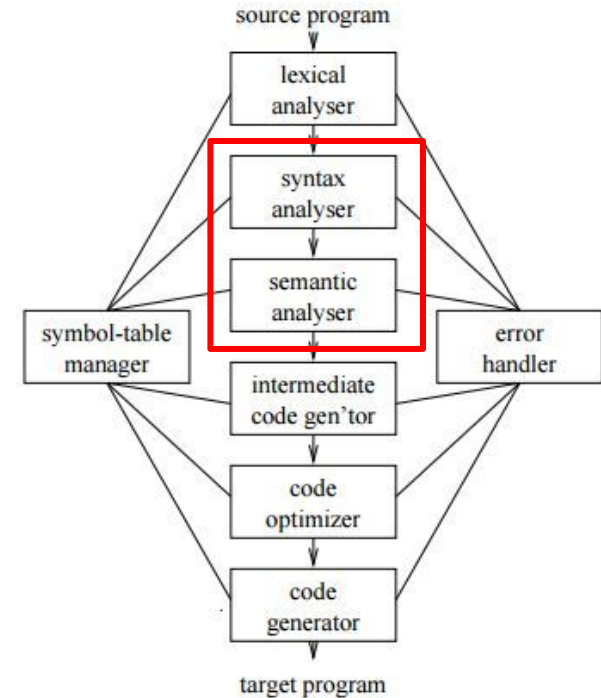
# Previously we have -

- Written Symbol Table
- Done lexical analysis (with Flex)



# In this assignment we will -

- Perform Syntax Analysis
- Perform Semantic Analysis
- We will use Antlr
  - Specifically Antlr4



# Back to Theory of Computation

- A language needs some syntax rules
  - That's what tells us “missing a semicolon after ...” when we forget semicolons in C
- Do you remember any techniques we learnt that governs the structure of some text?
  - Finite State Machines (Regular Languages)
  - Push Down Automaton (Context Free Languages)
  - Turing Machines (Recursively Enumerable or Recursive Languages)
- Almost all programming languages are **Context Free**
- So need to define **CFGs** when building a compiler for a language

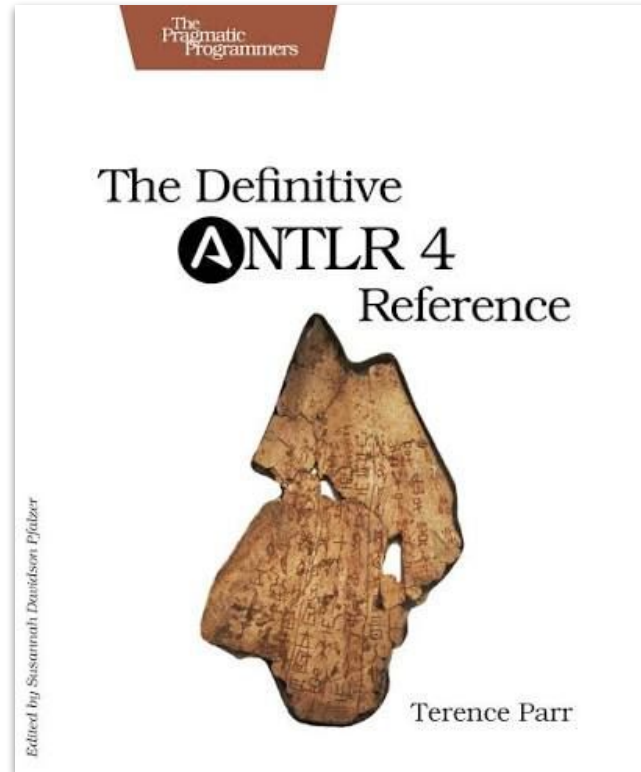
# What to do after defining the CFG?

- We need some mechanisms to parse a text
  - Check if the text follows the grammar or not
  - If it follows the grammar, then generate the corresponding **parse tree**
- Different algorithms exist
  - CYK algorithm
    - Global parser
    - But extremely slow
  - Top down parsing - LL
  - Bottom up parsing - LR
- Antlr uses a version of **LL algorithm**
  - **ALL(\*)**

# Antlr's internals - ALL(\*)

- “A” stands for Adaptive
  - Antlr does not create a static decision table
  - It decides which rule to take purely in **runtime**
- The first “L” stands for left to right scanning
- The second “L” stands for leftmost derivation
- The “ \* ” stands for infinite lookahead
  - To resolve ambiguities, Antlr may look ahead up until the end of the input, if necessary

For those who want to learn more



# How do I install Antlr?

- Working with Antlr needs 2 things -
  - The Antlr runtime
  - Antlr tools
- First we will install the runtime



# Antlr runtime installation

```
git clone https://github.com/Irtiaz/antlr4.git
cd antlr4
cd runtime/Cpp
mkdir build && cd build
cmake ..
make
sudo make install
```

# Antlr tool installation

- First create and enter a python virtual environment
- Then run **pip install antlr4-tools**
- Then run **antlr4 -v 4.13.2** to finally download and install the tool

# What Antlr does

- Antlr takes an antlr recognizable grammar file (.g4)
- Generates ALL(\*) parser code
- There are 3 patterns for using Antlr
  - On the fly (action embedded rules)
  - Visitor Pattern
  - Listener Pattern
- Visitor and Listener patterns are cleaner and easier to write
- They work **after** producing the parse tree

# Visitor vs Listener

- Both works **after** generating the parse tree
- Listener pattern only lets you *listen*
  - Antlr does the parse tree traversal with DFS
  - You simply **register listen callbacks** that Antlr calls
- Visitor pattern lets you *visit* however you want
  - You call the `visit()` functions in any order
    - Not necessarily DFS
- In our assignment, the **visitor pattern** is preferred

Enough with the theory!  
Let's dive in!!